

TALLER EVALUACIÓN DE RENDIMIENTO .

SISTEMAS OPERATIVOS .

INTEGRANTES:

ALEJANDRO BELTRAN HUERTAS

ANDRES DIAZ

MAURICIO BELTRÁN HUERTAS



Pontificia Universidad
JAVERIANA
Colombia

2025

3.1 Compilar los ficheros fuente

```
[(base) mauriciobeltranhuertas@MacBook-Pro-de-Mauricio-2 fuentes % make clean
rm -f mmClasicaFork mmClasicaPosix mmClasicaOpenMP mmFilasOpenMP
[(base) mauriciobeltranhuertas@MacBook-Pro-de-Mauricio-2 fuentes % make all
/opt/homebrew/bin/gcc-15 -lm mmClasicaFork.c -o mmClasicaFork
/opt/homebrew/bin/gcc-15 -lm mmClasicaPosix.c -o mmClasicaPosix -lpthread
/opt/homebrew/bin/gcc-15 -lm mmClasicaOpenMP.c -o mmClasicaOpenMP -fopenmp -O3
/opt/homebrew/bin/gcc-15 -lm mmFilasOpenMP.c -o mmFilasOpenMP -fopenmp -O3
(base) mauriciobeltranhuertas@MacBook-Pro-de-Mauricio-2 fuentes % █
```

Antes de realizar las pruebas de rendimiento, se compiló todos los programas fuente desarrollados durante el taller. Para esto, se utilizó el archivo Makefile, el cual automatiza el proceso de compilación y permite generar los ejecutables de manera ordenada, evitando tener que compilar manualmente cada archivo.

Primero se ejecutó el comando *make clean*, cuya función es eliminar los ejecutables generados previamente, en este caso: mmClasicaFork, mmClasicaPosix, mmClasicaOpenMP y mmFilasOpenMP. Esto garantiza que la compilación se realice desde cero, evitando conflictos.

Después de esto, se utilizó el comando *make all*, que compila todos los programas que están en el Makefile teniendo en cuenta las banderas necesarias según el modelo de paralelismo utilizado.

Como resultado, se generaron correctamente los cuatro ejecutables en la carpeta fuentes. Este proceso confirma que el Makefile funciona de la mejor manera, esto hace que se automatice la compilación de las diferentes versiones del algoritmo y facilitando la ejecución de los experimentos posteriores.

3.2 Ejecutar para valores de matrices menores a 5x5

```
[(base) mauriciobeltranhuertas@MacBook-Pro-de-Mauricio-2 fuentes % ./mmClasicaOpenMP 4 1
```

```
3.47 2.13 1.63 2.75
1.65 0.78 2.61 1.44
3.12 1.63 1.12 2.44
0.74 0.09 1.09 2.90
**-----**
```

```
1.69 2.09 2.82 0.60
5.89 3.94 5.80 3.74
1.89 0.59 6.10 3.22
1.11 5.49 6.11 2.13
**-----**
111
```

```
24.53 31.70 48.86 21.17
13.93 16.03 33.93 15.41
19.67 27.00 39.98 16.78
7.06 18.49 27.01 10.50
**-----**
```

3.3 Ejemplo por consola de ejecución para tamaño 4x4 y número de hilos 1

Para comprobar que el programa funciona correctamente, se hizo una prueba con el ejecutable mmClasicaOpenMP(punto 3.2) , usando una matriz de 4x4 y 1 solo hilo. Esto permite ver si la multiplicación de matrices se hace bien antes de usar más hilos. Al correr el comando `./mmClasicaOpenMP 4 1`, el programa genera dos matrices aleatorias A y B y luego muestra la matriz resultado $C = A \times B$. En la salida se puede ver claramente cómo los valores cambian según los números generados, confirmando que el cálculo se realiza correctamente.

3.5 Diseño del experimento.

El propósito de esta fase es planificar y ejecutar una batería de pruebas que permita evaluar el rendimiento de los distintos algoritmos de multiplicación de matrices implementados en las versiones *Fork*, *Pthreads*, *OpenMP* y *Filas OpenMP*.

3.5.1 Objetivo

El diseño experimental se basa en el análisis de variables de entrada, como el tamaño de la matriz y el número de hilos de ejecución, con el objetivo de determinar cómo influye el nivel de concurrencia en el tiempo total de procesamiento. De esta manera, se busca comparar la eficiencia de las implementaciones paralelas frente a la ejecución secuencial.

3.5.2 Variables del experimento

En esta sección se definen las variables que intervienen en la evaluación del rendimiento de los algoritmos de multiplicación de matrices. Las pruebas se estructuraron con base en un diseño factorial, donde se varía tanto el tamaño de la matriz como el número de hilos de ejecución, registrando como métrica dependiente el tiempo promedio de ejecución.

Variables independientes:

Las variables independientes son las que el usuario puede modificar para observar su efecto sobre el rendimiento:

Variable	Descripción	Valores utilizados	Justificación
Tamaño de la matriz	Representa la dimensión de las matrices cuadradas a multiplicar.	2x2, 10x10 y 20x20.	algoritmo a medida que aumenta la carga computacional y el uso de memoria.
Número de hilos	Cantidad de hilos o procesos concurrentes empleados por el algoritmo.	1, 2, 4, 8, 16, 20	Evalúa la ganancia en rendimiento al aumentar el grado de paralelismo hasta el límite de los núcleos físicos disponibles.

En este caso, las variables independientes corresponden al tamaño de la matriz y al número de hilos empleados durante la ejecución de los algoritmos de multiplicación de matrices.

Para el diseño experimental se eligieron diferentes tamaños de matrices con el propósito de analizar el comportamiento del rendimiento y la escalabilidad de cada modelo de paralelismo (Fork, POSIX y OpenMP).

Los tamaños seleccionados fueron 2×2 , 10×10 y 20×20 , ya que permiten observar tanto ejecuciones pequeñas donde se aprecia la sobrecarga inicial de creación de hilos o procesos, como ejecuciones más grandes, donde el acceso a memoria principal tiene un impacto significativo en el tiempo total de ejecución.

El tamaño 20×20 , en particular, introduce una mayor demanda de operaciones aritméticas y de memoria, lo que posibilita evaluar la eficiencia de la concurrencia y el aprovechamiento de la jerarquía de memoria del sistema.

Esta selección equilibra el tiempo de cómputo y el costo de sincronización entre hilos, proporcionando una base sólida para comparar los modelos de paralelismo bajo las mismas condiciones de hardware.

En cuanto al número de hilos, se utilizaron los valores 1, 2, 4, 8, 16 y 20, con el objetivo de analizar cómo varía el tiempo de ejecución a medida que aumenta el nivel de concurrencia.

Estas configuraciones permiten observar desde un escenario secuencial (1 hilo), pasando por niveles intermedios (2 y 4 hilos), hasta configuraciones de alta concurrencia (8, 16 y 20 hilos).

De esta forma, los resultados permiten evidenciar cómo el rendimiento mejora con el aumento de hilos hasta cierto punto óptimo, y cómo a partir de ese límite puede observarse la sobrecarga de sincronización o la saturación de los núcleos físicos disponibles.

Variables dependientes:

Las variables dependientes son aquellas que no se controlan directamente, sino que cambian como resultado de las modificaciones hechas por el usuario a las variables independientes.

Variable	Descripción	Unidad de medida	Método de recolección
Tiempo de ejecución (T)	Mide la duración total del procesamiento de la multiplicación de matrices para cada configuración experimental.	Microsegundos	Calculado automáticamente dentro del código mediante las funciones InicioMuestra() y FinMuestra(), que usan la estructura timeval para medir el tiempo real transcurrido.

Variables Controladas:

Las variables controladas son aquellas que se mantienen constantes durante todo el experimento para evitar que afecten los resultados.

Variable	Descripción	Forma de control
Hardware y sistema operativo	Las pruebas se realizaron en el mismo equipo de cómputo para evitar variaciones externas.	Ejecución en un solo sistema físico.
Compilador y optimización	Se mantuvo el mismo compilador y nivel de optimización en todas las pruebas.	Controlado mediante el mismo Makefile para todas las versiones.
Cantidad de repeticiones	Cada combinación de tamaño e hilos se ejecutó 30 veces para obtener un promedio representativo.	Controlado en el script lanzador.pl mediante la variable \$Repeticiones = 30;.

En conclusión, las variables seleccionadas permiten evaluar de forma controlada la influencia del tamaño de la matriz y del número de hilos sobre el rendimiento del algoritmo, garantizando resultados comparables y reproducibles.

3.5.3 Diseño factorial

El diseño factorial del experimento permite analizar de forma sistemática la interacción entre las variables independientes: tamaño de la matriz y número de hilos de ejecución.

Cada combinación de estas variables representa una configuración experimental distinta, la cual se ejecuta repetidamente para obtener resultados estadísticamente representativos.

El propósito de este diseño es evaluar el efecto individual y combinado de ambas variables sobre la variable dependiente, garantizando que todas las combinaciones sean probadas bajo las mismas condiciones controladas.

Para este caso se plantearon las siguientes combinaciones experimentales

Tamaño de matriz	Número de hilos	Número de repeticiones por configuración
2x2	1, 2, 4, 8, 16, 20	30
10x10	1, 2, 4, 8, 16, 20	30
20x20	1, 2, 4, 8, 16, 20	30

SISTEMA #1

Las pruebas experimentales del taller se realizaron en un MacBook Pro de 14 pulgadas modelo 2023, equipado con el chip Apple M3 Pro. Este equipo fue seleccionado por su capacidad para ejecutar tareas de cómputo intensivo aprovechando la arquitectura de múltiples núcleos del procesador M-Series, lo que permite comparar de manera efectiva los diferentes enfoques de paralelismo implementados: Fork, POSIX, OpenMP clásico y OpenMP por filas.

A continuación se presentan las principales características del sistema de cómputo utilizado:

- Equipo: MacBook Pro, 14 pulgadas, 2023
- Chip: Apple M3 Pro, arquitectura ARM64
- CPU: 11 núcleos, 5 de rendimiento + 6 de eficiencia
- Memoria RAM: 18 GB LPDDR5
- Sistema operativo: macOS Tahoe versión 26.0.1
- Pantalla: Liquid Retina XDR 14" 3024 × 1964 px
- Almacenamiento: SSD de 1 TB (aprox. 916 GB disponibles)
- Compilador utilizado: GCC 15 (instalado mediante Homebrew)
- Soporte de paralelismo: OpenMP y POSIX Threads habilitados

El equipo ofrece un entorno estable y potente para la ejecución de programas paralelos, aprovechando los recursos del chip M3 Pro para distribuir la carga de trabajo entre sus núcleos. Gracias a esto, fue posible comparar el comportamiento del rendimiento en los distintos modelos de paralelismo propuestos, evaluando el impacto del número de hilos y el tamaño de las matrices en los tiempos de ejecución.

3.5.4

```
[(base) mauriciobeltranhuertas@MacBook-Pro-de-Mauricio-2 fuentes % perl lanzador.pl

=== Ejecutando mmClasicaFork ===
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaFork 2 1
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaFork 2 2
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaFork 10 1
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaFork 10 2
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaFork 10 4
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaFork 10 8
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaFork 20 1
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaFork 20 2
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaFork 20 4
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaFork 20 8
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaFork 20 16
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaFork 20 20

=== Ejecutando mmClasicaPosix ===
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaPosix 2 1
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaPosix 2 2
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaPosix 10 1
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaPosix 10 2
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaPosix 10 4
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaPosix 10 8
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaPosix 20 1
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaPosix 20 2
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaPosix 20 4
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaPosix 20 8
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaPosix 20 16
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaPosix 20 20

=== Ejecutando mmClasicaOpenMP ===
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaOpenMP 2 1
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaOpenMP 2 2
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaOpenMP 10 1
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaOpenMP 10 2
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaOpenMP 10 4
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaOpenMP 10 8
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaOpenMP 20 1
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaOpenMP 20 2
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaOpenMP 20 4
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaOpenMP 20 8
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaOpenMP 20 16
/Users/mauriciobeltranhuertas/downloads/fuentes/mmClasicaOpenMP 20 20

=== Ejecutando mmFilasOpenMP ===
/Users/mauriciobeltranhuertas/downloads/fuentes/mmFilasOpenMP 2 1
/Users/mauriciobeltranhuertas/downloads/fuentes/mmFilasOpenMP 2 2
/Users/mauriciobeltranhuertas/downloads/fuentes/mmFilasOpenMP 10 1
/Users/mauriciobeltranhuertas/downloads/fuentes/mmFilasOpenMP 10 2
/Users/mauriciobeltranhuertas/downloads/fuentes/mmFilasOpenMP 10 4
/Users/mauriciobeltranhuertas/downloads/fuentes/mmFilasOpenMP 10 8
/Users/mauriciobeltranhuertas/downloads/fuentes/mmFilasOpenMP 20 1
/Users/mauriciobeltranhuertas/downloads/fuentes/mmFilasOpenMP 20 2
/Users/mauriciobeltranhuertas/downloads/fuentes/mmFilasOpenMP 20 4
/Users/mauriciobeltranhuertas/downloads/fuentes/mmFilasOpenMP 20 8
/Users/mauriciobeltranhuertas/downloads/fuentes/mmFilasOpenMP 20 16
/Users/mauriciobeltranhuertas/downloads/fuentes/mmFilasOpenMP 20 20
(base) mauriciobeltranhuertas@MacBook-Pro-de-Mauricio-2 fuentes % █
```

Durante esta etapa se ejecutó el script de automatización lanzador.pl, encargado de realizar todas las pruebas de manera ordenada y repetitiva. El script recorre los cuatro programas mmClasicaFork, mmClasicaPosix, mmClasicaOpenMP y mmFilasOpenMP, ejecutándolos para los distintos tamaños de matriz definidos 2, 10 y 20 y para los diferentes números de hilos 1, 2, 4, 8, 16 y 20. De esta manera se logró cubrir todas las combinaciones posibles entre el tamaño de la matriz y el nivel de concurrencia, garantizando un análisis completo del rendimiento en cada modelo de paralelismo.

Además, el script automatizó la repetición de 30 ejecuciones por configuración, almacenando los resultados de tiempo en archivos individuales con formato .dat. Esto permitió obtener

datos más confiables y reducir la variabilidad en las mediciones, ya que los tiempos reportados reflejan el comportamiento promedio del sistema y no una única ejecución aislada.

3.5.5

Implementación	Tamaño	Hilos	Promedio (μs)
mmFilasOpenMP	20	20	150
mmFilasOpenMP	20	16	122,33333333
mmFilasOpenMP	20	8	74,36666667
mmFilasOpenMP	20	4	39,53333333
mmFilasOpenMP	20	2	25,43333333
mmFilasOpenMP	20	1	6,93333333
mmFilasOpenMP	10	20	147,06666667
mmFilasOpenMP	10	16	124,73333333
mmFilasOpenMP	10	8	69,3
mmFilasOpenMP	10	4	39,8
mmFilasOpenMP	10	2	27,66666667
mmFilasOpenMP	10	1	5,53333333
mmFilasOpenMP	2	20	149,33333333
mmFilasOpenMP	2	16	124,2
mmFilasOpenMP	2	8	70,36666667
mmFilasOpenMP	2	4	40,53333333
mmFilasOpenMP	2	2	26,93333333
mmFilasOpenMP	2	1	4,9

Análisis de resultados

Al revisar los resultados, se nota que cuando se aumentan los hilos, el tiempo de ejecución también sube, lo que significa que en este caso el paralelismo no está ayudando al rendimiento, sino que lo empeora. En lugar de acelerar el proceso, el uso de más hilos genera una sobrecarga adicional por la creación, coordinación y sincronización entre ellos.

Por ejemplo, con matrices pequeñas de 2x2 el tiempo pasa de 4,9 μs con 1 hilo a 149,3 μs con 20 hilos, y algo similar ocurre con las de 10x10 y 20x20, donde los tiempos también aumentan a medida que crece el número de hilos. Esto demuestra que las matrices que se usaron son demasiado pequeñas como para aprovechar la concurrencia. En pocas palabras, el trabajo que implica manejar tantos hilos termina siendo mayor que el beneficio que se obtiene al dividir las operaciones. Por eso, el mejor rendimiento se da con uno o pocos hilos, donde el sistema trabaja de manera más directa y sin tanta carga de sincronización.

Implementación	Tamaño	Hilos	Promedio (μs)
mmClasicaOpenMP	20	20	148,2333333
mmClasicaOpenMP	20	16	125,6
mmClasicaOpenMP	20	8	70,86666667
mmClasicaOpenMP	20	4	40,8
mmClasicaOpenMP	20	2	27,93333333
mmClasicaOpenMP	20	1	6,766666667
mmClasicaOpenMP	10	20	148,5
mmClasicaOpenMP	10	16	122,3666667
mmClasicaOpenMP	10	8	70,53333333
mmClasicaOpenMP	10	4	38,36666667
mmClasicaOpenMP	10	2	26,9
mmClasicaOpenMP	10	1	4,466666667
mmClasicaOpenMP	2	20	144,6
mmClasicaOpenMP	2	16	120,3666667
mmClasicaOpenMP	2	8	68,03333333
mmClasicaOpenMP	2	4	38,93333333
mmClasicaOpenMP	2	2	26,53333333
mmClasicaOpenMP	2	1	4,4

Análisis de resultados

Al analizar los resultados de la implementación mmClásicaOpenMP, se puede ver una tendencia muy parecida a la de la versión por filas: cuando se incrementa el número de hilos, el tiempo de ejecución también sube. Esto quiere decir que el uso del paralelismo en este caso no mejora el rendimiento, sino que lo empeora debido a la sobrecarga que genera coordinar los hilos.

Por ejemplo, con una matriz pequeña de 2x2, el tiempo pasa de 4,4 μs con 1 hilo a 144,6 μs con 20 hilos. Lo mismo ocurre con las matrices de 10x10 y 20x20, donde los tiempos aumentan progresivamente al usar más hilos. Esto demuestra que las matrices utilizadas son muy pequeñas para aprovechar realmente el trabajo paralelo. En este escenario, el costo de crear y sincronizar los hilos termina siendo más alto que el beneficio obtenido. En conclusión, el mejor desempeño se da con uno o dos hilos, donde el procesamiento es más directo y no hay tanta sobrecarga de comunicación entre los hilos.

Implementación	Tamaño	Hilos	Promedio (μs)
mmClasicaPosix	20	20	190,9
mmClasicaPosix	20	16	144,63333333
mmClasicaPosix	20	8	69,933333333
mmClasicaPosix	20	4	40,3
mmClasicaPosix	20	2	31,16666667
mmClasicaPosix	20	1	30,96666667
mmClasicaPosix	10	20	186,3
mmClasicaPosix	10	16	146,9
mmClasicaPosix	10	8	67,6
mmClasicaPosix	10	4	36
mmClasicaPosix	10	2	27,2
mmClasicaPosix	10	1	21,7
mmClasicaPosix	2	20	180,9
mmClasicaPosix	2	16	146,7666667
mmClasicaPosix	2	8	70,1
mmClasicaPosix	2	4	36,233333333
mmClasicaPosix	2	2	27,333333333
mmClasicaPosix	2	1	19,933333333

Análisis de resultados

En la implementación mmClásicaPosix también se observa un patrón similar a los casos anteriores: al aumentar la cantidad de hilos, los tiempos de ejecución suben en lugar de bajar. Esto deja claro que el uso de múltiples hilos no está ayudando al rendimiento, sino que está generando más carga de coordinación entre los procesos.

Por ejemplo, con una matriz pequeña de 2x2, el tiempo pasa de 19,93 μs con 1 hilo a 180,9 μs con 20 hilos, y con matrices de 10x10 y 20x20 se ve la misma tendencia: mientras más hilos se usan, mayor es el tiempo promedio. Esto demuestra que el tamaño de las matrices no es suficiente para aprovechar la paralelización con POSIX. En este caso, crear y manejar tantos hilos termina costando más que hacer la multiplicación de forma secuencial. En conclusión, la mejor eficiencia se obtiene con pocos hilos (1 o 2), ya que la sobrecarga de sincronización y creación de hilos afecta significativamente el rendimiento cuando se usan más.

Implementación	Tamaño	Hilos	Promedio (μ s)
mmClasicaFork	20	20	4587,733333
mmClasicaFork	20	16	3785,566667
mmClasicaFork	20	8	1879,1
mmClasicaFork	20	4	981,5
mmClasicaFork	20	2	503,5
mmClasicaFork	20	1	320,5
mmClasicaFork	10	20	4582,266667
mmClasicaFork	10	16	3734,566667
mmClasicaFork	10	8	1834,566667
mmClasicaFork	10	4	962,4
mmClasicaFork	10	2	496,7
mmClasicaFork	10	1	314,8666667
mmClasicaFork	2	20	
mmClasicaFork	2	16	
mmClasicaFork	2	8	
mmClasicaFork	2	4	1135,206897
mmClasicaFork	2	2	677,1333333
mmClasicaFork	2	1	478,8

Análisis de resultados

En la implementación mmClásicaFork, los resultados muestran claramente que el rendimiento empeora de forma significativa al aumentar el número de procesos. En este caso, el tiempo de ejecución se incrementa de manera drástica, lo que evidencia una gran sobrecarga en la creación y gestión de procesos.

Por ejemplo, con una matriz de 2x2, el tiempo sube de 478,8 μ s con 1 proceso a más de 1135 μ s con 4 procesos. En el caso de las matrices más grandes 10x10 y 20x20, los tiempos alcanzan valores muy altos, superando los 4500 μ s al usar 20 procesos. Esto demuestra que el uso de la función fork() no es eficiente para este tipo de operaciones, ya que crear procesos independientes implica copiar el espacio de memoria, lo cual consume muchos más recursos que el uso de hilos. En conclusión, el paralelismo con Fork no mejora el rendimiento y solo genera más carga para el sistema, por lo que la versión secuencial con un solo proceso sigue siendo la más eficiente en este caso.

3.5.6

Para la automatización de la batería de experimentos se utilizó el archivo lanzador.pl, un script desarrollado en Perl que permite ejecutar de manera continua y sistemática todas las combinaciones de tamaños de matriz y número de hilos definidos en la batería de pruebas. Este script se ejecutó con el comando: “./[lanzador.pl](#)”

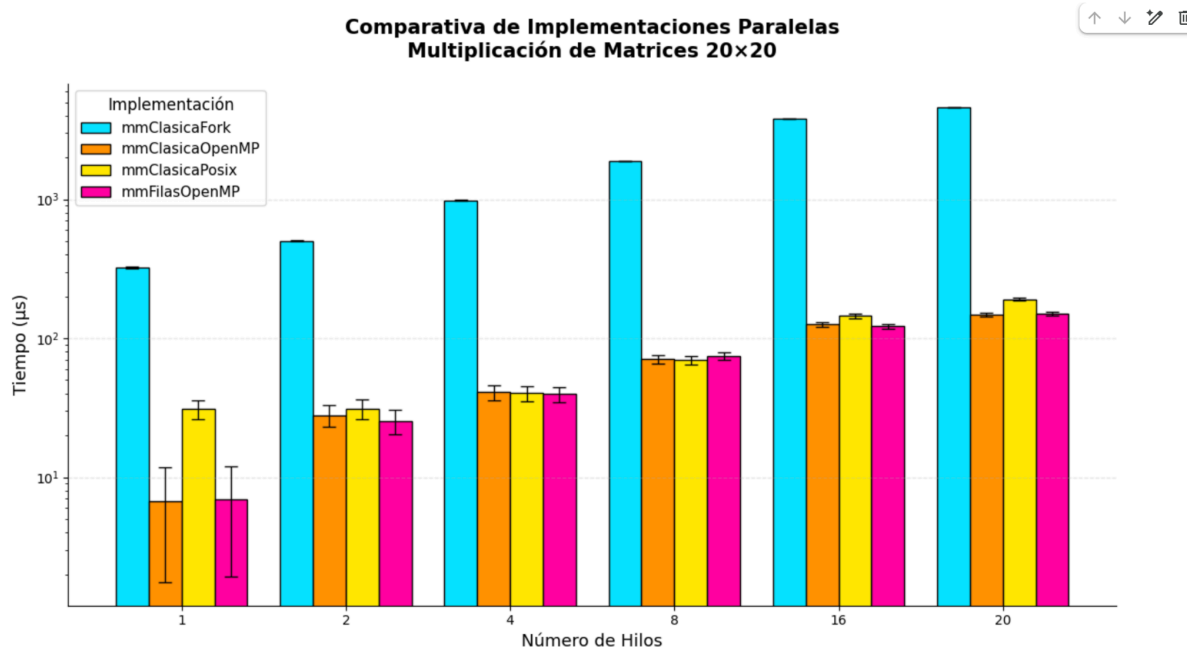
El objetivo principal fue evitar la ejecución manual de cada prueba y garantizar que todas las configuraciones se corrieran bajo las mismas condiciones. El fichero lanzador.pl fue modificado para capturar los tiempos de ejecución de cada implementación Fork, POSIX,

OpenMP Clásica y OpenMP por Filas. Y así repetir cada caso 30 veces, calculando un promedio que luego se almacenó automáticamente en archivos .dat.



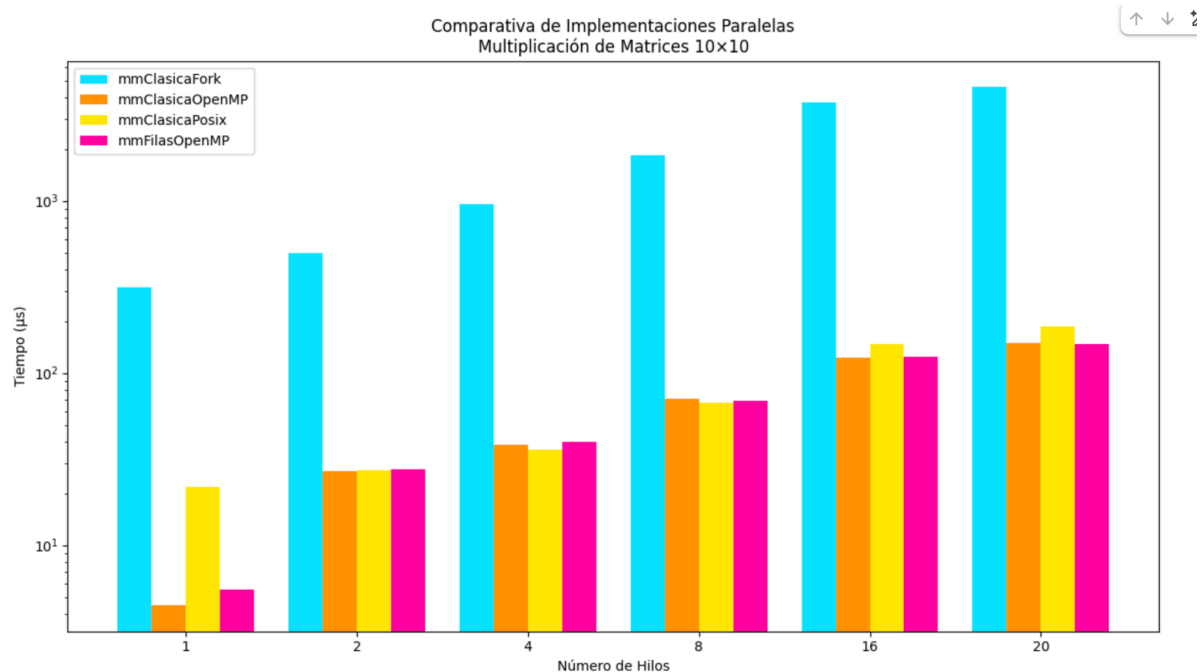
Por último, estos archivos se exportaron a Excel para realizar el análisis comparativo de rendimiento y construir las tablas correspondientes. Este proceso automatizado permitió obtener datos más precisos, minimizar errores humanos y facilitar la observación de tendencias en el comportamiento de cada implementación frente a diferentes niveles de paralelismo.

3.5.7



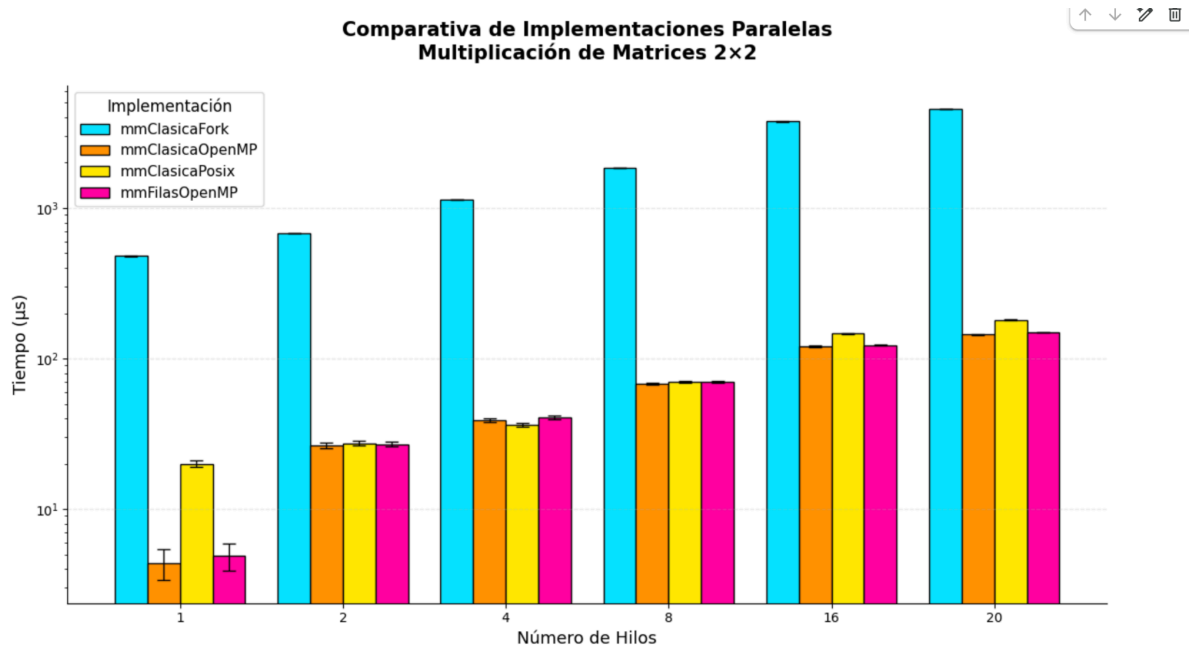
Para las matrices 20×20, el comportamiento general se mantiene: el tiempo crece con el número de hilos.

No se produce aceleración paralela porque la carga sigue siendo demasiado baja para compensar el costo de coordinación entre hilos. Sin embargo, mmFilasOpenMP muestra un crecimiento más estable, ligeramente mejor que OpenMP clásico y POSIX, gracias a una mayor localización espacial de memoria. Aun así, el mejor tiempo siempre se obtiene con 1 hilo, y Fork continúa siendo el más ineficiente.



En las matrices 10×10 , aunque hay más carga computacional, el comportamiento sigue siendo el mismo:
más hilos = más tiempo.

El overhead de paralelización no se amortiza con matrices pequeñas, por lo que no se observa ninguna mejora de rendimiento. OpenMP vuelve a ser la opción más eficiente, POSIX tiene un comportamiento intermedio, y Fork es consistentemente el peor. Esto confirma que, para cargas moderadas, la concurrencia empeora el rendimiento.



En las matrices 2×2 , el paralelismo no aporta ningún beneficio.

Todas las implementaciones aumentan su tiempo conforme crecen los hilos, lo cual confirma que el costo de crear, coordinar y sincronizar hilos supera por completo el costo computacional de multiplicar una matriz tan pequeña. OpenMP clásico y por filas son los más eficientes en valores absolutos, mientras que Fork es el menos eficiente debido al alto costo de creación de procesos. La versión secuencial con un solo hilo sigue siendo la opción más rápida.

SISTEMA #2

Para el segundo sistema se utilizó un computador con estas características

Parámetro	Descripción
Modelo	MacBook Pro 14
Chip (CPU)	Apple M3 Pro – arquitectura ARM64
Número de núcleos / hilos	12 núcleos totales
Memoria RAM	18 GB
Sistema operativo	macOS Tahoe versión 26.0.1
Compilador	GCC 15.2.0 (Homebrew), con soporte para OpenMP y POSIX Threads (gcc-15 --version)
Lenguaje y librerías	C estándar con librerías pthreads y OpenMP
Script de automatización	Perl v5.34.1
Arquitectura	ARM64

Los resultados que se obtuvieron fueron estos para el algoritmo mmClasicaFork:

Programa	Tamaño de matriz	Nº de hilos	Tiempo (µs)
mmClasicaFork	2x2	1	397
mmClasicaFork	2x2	2	530,4
mmClasicaFork	2x2	4	968,73333
mmClasicaFork	2x2	8	1905,7
mmClasicaFork	2x2	16	3715,0333
mmClasicaFork	2x2	20	4585,4333
mmClasicaFork	10x10	1	305,8
mmClasicaFork	10x10	2	536,33333
mmClasicaFork	10x10	4	976,53333
mmClasicaFork	10x10	8	1.882
mmClasicaFork	10x10	16	3669,1
mmClasicaFork	10x10	20	4583,5
mmClasicaFork	20x20	1	318
mmClasicaFork	20x20	2	519,76667
mmClasicaFork	20x20	4	950,03333
mmClasicaFork	20x20	8	1907,1667
mmClasicaFork	20x20	16	3750,5
mmClasicaFork	20x20	20	4597,6667

La tabla de mmClasicaFork evidencia que esta implementación basada en procesos es, por mucho, la menos eficiente entre todas las pruebas realizadas. Los tiempos de ejecución crecen casi de manera lineal a medida que aumenta el número de procesos, sin importar el tamaño de la matriz. Incluso con una matriz mínima como 2×2, los tiempos se ubican entre 397 µs y más de 4500 µs, lo que demuestra un costo excesivo en la creación y gestión de procesos. De hecho, los tiempos prácticamente no varían de manera proporcional al tamaño de la matriz, lo cual indica que la mayor parte del tiempo consumido proviene del overhead asociado al fork() y no del cálculo matemático. Esta implementación resulta completamente

ineficiente para tareas pequeñas o medianas y no presenta ningún beneficio derivado del paralelismo.

Los resultados que se obtuvieron fueron estos para el algoritmo mmClasicaOpenMP:

Programa	Tamaño de matriz	Nº de hilos	Tiempo (μ s)
mmClasicaOpenMP	2x2	1	10,133333
mmClasicaOpenMP	2x2	2	25,466667
mmClasicaOpenMP	2x2	4	40
mmClasicaOpenMP	2x2	8	78,466667
mmClasicaOpenMP	2x2	16	141,76667
mmClasicaOpenMP	2x2	20	183,6
mmClasicaOpenMP	10x10	1	4,5333333
mmClasicaOpenMP	10x10	2	27,733333
mmClasicaOpenMP	10x10	4	40,566667
mmClasicaOpenMP	10x10	8	74
mmClasicaOpenMP	10x10	16	132,6
mmClasicaOpenMP	10x10	20	152,26667
mmClasicaOpenMP	20x20	1	6,7
mmClasicaOpenMP	20x20	2	28,8
mmClasicaOpenMP	20x20	4	39,833333
mmClasicaOpenMP	20x20	8	74,266667
mmClasicaOpenMP	20x20	16	134,16667
mmClasicaOpenMP	20x20	20	164,36667

La tabla de mmClasicaOpenMP muestra una implementación mucho más eficiente que la versión con procesos. Para todos los tamaños de matriz, los tiempos se mantienen en rangos muy bajos, desde 10 μ s hasta aproximadamente 180 μ s. Sin embargo, la tendencia principal es que el tiempo de ejecución aumenta a medida que se incrementa el número de hilos, lo cual indica que el overhead del paralelismo supera el beneficio cuando la carga computacional es pequeña. La consistencia de los resultados a lo largo de los tres tamaños de matriz sugiere que OpenMP administra correctamente los hilos, pero aun así el costo de coordinación y sincronización domina el tiempo total. Es una implementación rápida, pero no es capaz de aprovechar el paralelismo con matrices tan pequeñas.

Los resultados que se obtuvieron fueron estos para el algoritmo mmClasicaPosix:

Programa	Tamaño de matriz	Nº de hilos	Tiempo (µs)
mmClasicaPosix	2x2	1	26,833333
mmClasicaPosix	2x2	2	24,9
mmClasicaPosix	2x2	4	34,6
mmClasicaPosix	2x2	8	66,8
mmClasicaPosix	2x2	16	145,56667
mmClasicaPosix	2x2	20	182,1
mmClasicaPosix	10x10	1	20,866667
mmClasicaPosix	10x10	2	27,233333
mmClasicaPosix	10x10	4	37,066667
mmClasicaPosix	10x10	8	71
mmClasicaPosix	10x10	16	150,06667
mmClasicaPosix	10x10	20	185,66667
mmClasicaPosix	20x20	1	23,733333
mmClasicaPosix	20x20	2	30,1
mmClasicaPosix	20x20	4	39,366667
mmClasicaPosix	20x20	8	70,233333
mmClasicaPosix	20x20	16	148,26667
mmClasicaPosix	20x20	20	187,33333

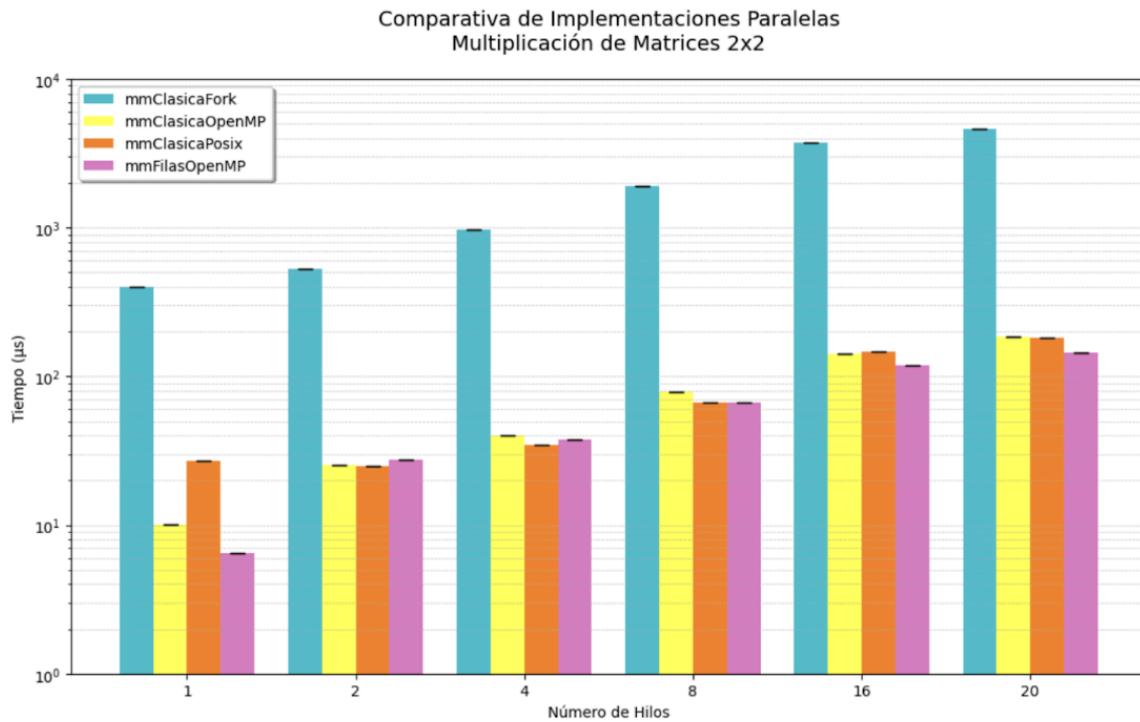
Los datos obtenidos con el algoritmo mmClasicaPosix evidencian un comportamiento muy similar al de OpenMP clásico. Los tiempos se mantienen entre 20 µs y 185 µs aproximadamente, dependiendo del número de hilos, y al igual que en OpenMP, el rendimiento disminuye a medida que se agregan más hilos. POSIX threads muestra pequeñas variaciones adicionales, especialmente frente a OpenMP, debido a que en este caso la configuración y sincronización dependen directamente de la implementación manual del programador. A pesar de estas variaciones, POSIX mantiene tiempos competitivos y coherentes, aunque nunca logra superar de forma consistente al rendimiento de OpenMP. Esta implementación es eficiente, pero tampoco se ve beneficiada por el aumento de hilos debido al escaso tamaño de las tareas.

Los resultados que se obtuvieron fueron estos para el algoritmo mmFilasOpenMP:

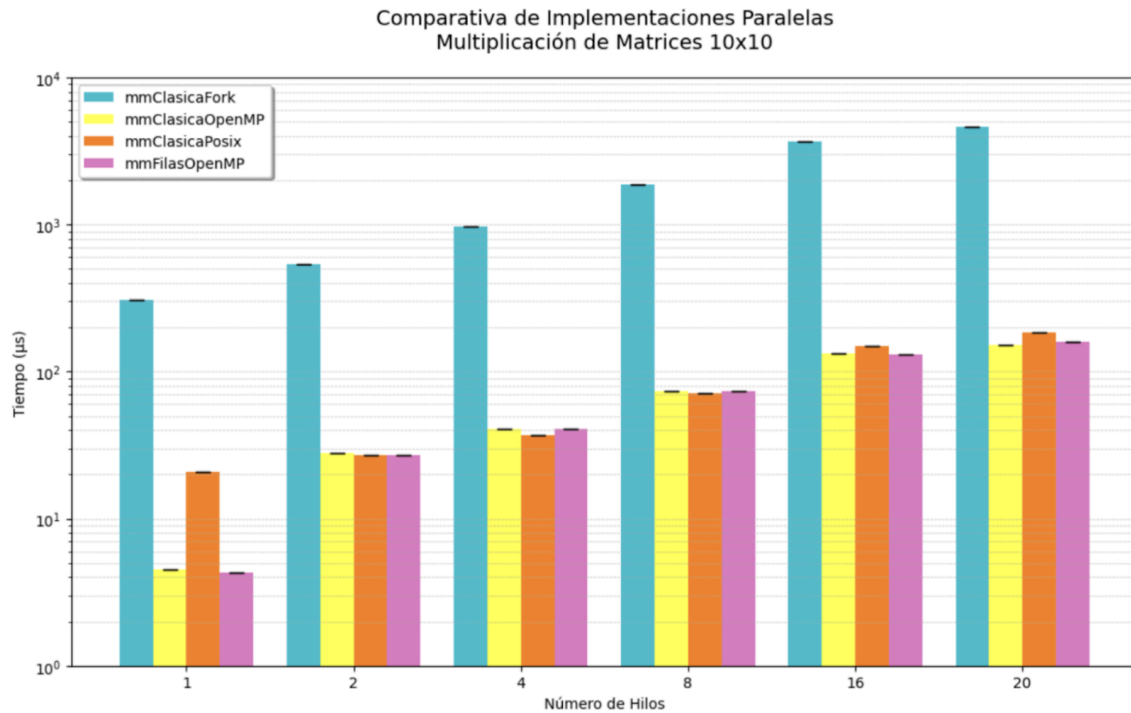
Programa	Tamaño de matriz	Nº de hilos	Tiempo (µs)
mmFilasOpenMP	2x2	1	6,5
mmFilasOpenMP	2x2	2	27,366667
mmFilasOpenMP	2x2	4	37,433333
mmFilasOpenMP	2x2	8	66,3
mmFilasOpenMP	2x2	16	117,96667
mmFilasOpenMP	2x2	20	143,8
mmFilasOpenMP	10x10	1	4,2666667
mmFilasOpenMP	10x10	2	27,166667
mmFilasOpenMP	10x10	4	41,066667
mmFilasOpenMP	10x10	8	74
mmFilasOpenMP	10x10	16	130,53333
mmFilasOpenMP	10x10	20	158,83333
mmFilasOpenMP	20x20	1	6,4666667
mmFilasOpenMP	20x20	2	29,466667
mmFilasOpenMP	20x20	4	42
mmFilasOpenMP	20x20	8	70,933333
mmFilasOpenMP	20x20	16	133,66667
mmFilasOpenMP	20x20	20	158,46667

La tabla muestra la implementación con mejor rendimiento relativo entre las estrategias basadas en hilos. Sus tiempos son consistentemente más bajos que los de OpenMP clásico y POSIX, especialmente cuando se utiliza un número reducido de hilos. Esto se debe a que la distribución del trabajo por filas reduce la sobrecarga y organiza el cálculo de manera más natural y balanceada. Aunque la tendencia se mantiene, esta versión presenta una estabilidad notable entre los distintos tamaños de matriz, y en varios casos obtiene los mejores tiempos absolutos. Por lo tanto la implementación más eficiente y equilibrada dentro del conjunto de soluciones paralelas evaluadas.

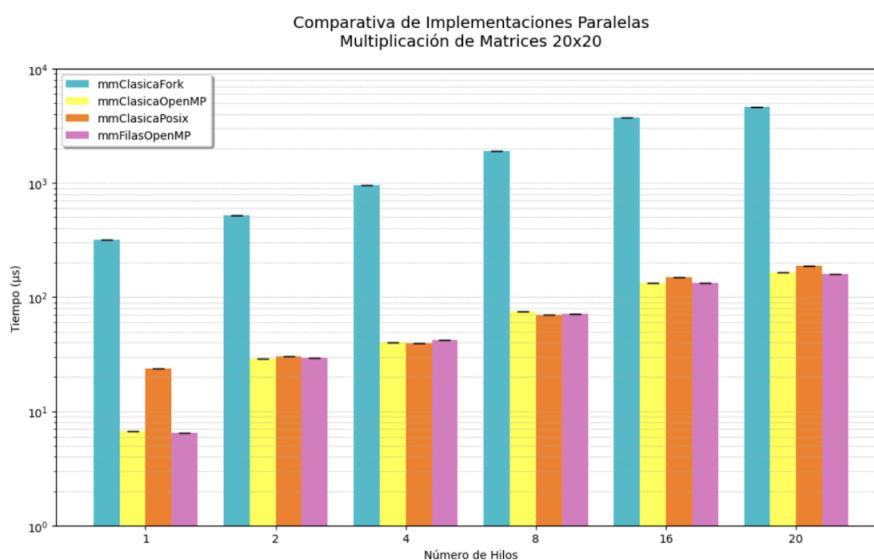
A continuación, se muestran las gráficas de rendimiento para cada tamaño de matriz, donde se comparan las cuatro implementaciones paralelas en función del número de hilos utilizados. Estas visualizaciones permiten identificar el comportamiento, la eficiencia y la sobrecarga asociada a cada técnica de paralelismo.



La gráfica de multiplicación de matrices 2×2 confirma visualmente que mmClasicaFork es completamente ineficiente, teniendo tiempos mucho más superiores que los demás. Mientras que fork alcanza miles de microsegundos, las variantes con hilos operan entre 5 μs y 180 μs. En todas las implementaciones con hilos, el tiempo de ejecución aumenta a medida que se incrementa el número de hilos, lo que demuestra claramente que para cargas extremadamente pequeñas el paralelismo no solo no aporta beneficios, sino que introduce overhead significativo. La diferencia entre OpenMP clásico, POSIX y FilasOpenMP es mínima debido a que la tarea es demasiado pequeña para aprovechar el paralelismo real.



La gráfica correspondiente a 10×10 es muy parecida a la grafica de 2×2 mmClasicaFork continúa mostrando tiempos desproporcionadamente altos, mientras que las implementaciones basadas en hilos se mantienen en un rango entre $4 \mu s$ y $160 \mu s$. La separación entre las variantes empieza a hacerse más visible por ejemplo FilasOpenMP suele presentar mejores tiempos, OpenMP clásico sigue cerca y POSIX se ubica un poco atras. Sin embargo, el cambio general persiste ya que todos los métodos paralelos empeoran su rendimiento cuando aumenta el número de hilos. Este comportamiento indica que, aun con un tamaño de matriz mayor, el trabajo sigue siendo insuficiente para compensar el costo del paralelismo.



En la gráfica de multiplicación 20×20 se confirma que mmClasicaFork sigue siendo la peor opción, con tiempos de miles de microsegundos. Las versiones con hilos muestran diferencias un poco más visibles, aunque todas se mantienen por debajo de $170 \mu s$. mmFilasOpenMP continúa siendo la más eficiente, seguida por OpenMP clásico y luego POSIX. Aun con matrices 20×20 , el aumento de hilos empeora el rendimiento debido al costo de sincronización y gestión, lo que muestra que este tamaño sigue siendo demasiado pequeño para obtener beneficios reales del paralelismo.

SISTEMA #3

Para el tercer sistema de cómputo se optó por un MacBook Air del 2015, con macOS Monterey y un software en la versión 12.7.6. Asimismo, este dispositivo cuenta con 1.6 GHz, un procesador Intel core i5 de dos núcleos, en cuanto a memoria tiene 8 GB de RAM, con una rapidez de 1600 MHZ de la generación DDR3 (double data rate). Por otra parte, cuenta con 190 GB disponibles de un total de 250 GB y un compilador GCC 13 instalado mediante MacPorts.

Archivo	Tamaño	Hilo	Promedio (μs)
MM_ClasicaFork	2	1	970,2
MM_ClasicaFork	2	2	1306,1
MM_ClasicaFork	10	1	988,1
MM_ClasicaFork	10	2	1160,8
MM_ClasicaFork	10	4	2265,3
MM_ClasicaFork	10	8	4095,2
MM_ClasicaFork	20	1	997
MM_ClasicaFork	20	2	1275,3
MM_ClasicaFork	20	4	2624
MM_ClasicaFork	20	8	4112,2
MM_ClasicaFork	20	16	8999,6
MM_ClasicaFork	20	20	10095,4

En el caso de las matrices 2×2 y 10×10 , el uso de más hilos no genera una mejora e incluso puede aumentar el tiempo de ejecución. Esto ocurre porque el costo de creación y sincronización de procesos en el modelo Fork llega a ser mayor que el beneficio obtenido por realizar las operaciones en paralelo cuando el número de procedimientos es bajo.

Por otra parte, para matrices de tamaño 20×20 se observa que el tiempo de ejecución crece a medida que aumenta el número de hilos. Mostrando que en este computador y situación el paralelismo con Fork no maneja eficientemente este problema, posiblemente debido al overhead de gestión de procesos y al costo de comunicación entre ellos.

Archivo	Tamaño	Hilos	Promedio (µs)
MM_ClasicaOpenMP	2	1	17,1
MM_ClasicaOpenMP	2	2	93,02
MM_ClasicaOpenMP	10	1	16,93
MM_ClasicaOpenMP	10	2	107,34
MM_ClasicaOpenMP	10	4	159,3
MM_ClasicaOpenMP	10	8	281,3
MM_ClasicaOpenMP	20	1	19,97
MM_ClasicaOpenMP	20	2	117,73
MM_ClasicaOpenMP	20	4	158,13
MM_ClasicaOpenMP	20	8	291,67
MM_ClasicaOpenMP	20	16	506,63
MM_ClasicaOpenMP	20	20	514,18

En este caso, los tiempos de ejecución son mucho menores en comparación con el modelo Fork, especialmente con pocos hilos. Sin embargo, entre más hilos se utilizan mayor es el tiempo de ejecución indicando una perdida de rendimiento.

Para todas las matrices, 2x2, 10x10 y 20x20 al aumentar el número de hilos aumentar el tiempo promedio, esto se debe al costo de gestión y sincronización de los hilos con respecto al tamaño del problema ya que se consumen más recursos en las operaciones de los hilos que el beneficio que trae usar estos. Aun así, los tiempos son mucho más bajos mostrando que OpenMP es una mejor opción que Fork para esta tarea.

Archivo	Tamaño	Hilos	Promedio (µs)
MM_ClasicaPosix	2	1	85,3
MM_ClasicaPosix	2	2	128,43
MM_ClasicaPosix	10	1	95,47
MM_ClasicaPosix	10	2	103,53
MM_ClasicaPosix	10	4	177,43
MM_ClasicaPosix	10	8	297,1
MM_ClasicaPosix	20	1	125,83
MM_ClasicaPosix	20	2	128,53
MM_ClasicaPosix	20	4	182,93
MM_ClasicaPosix	20	8	333,21
MM_ClasicaPosix	20	16	591,23
MM_ClasicaPosix	20	20	795,6

Los resultados con el modelo POSIX muestran tiempos mayores que en OpenMP, pero menores que en Fork.

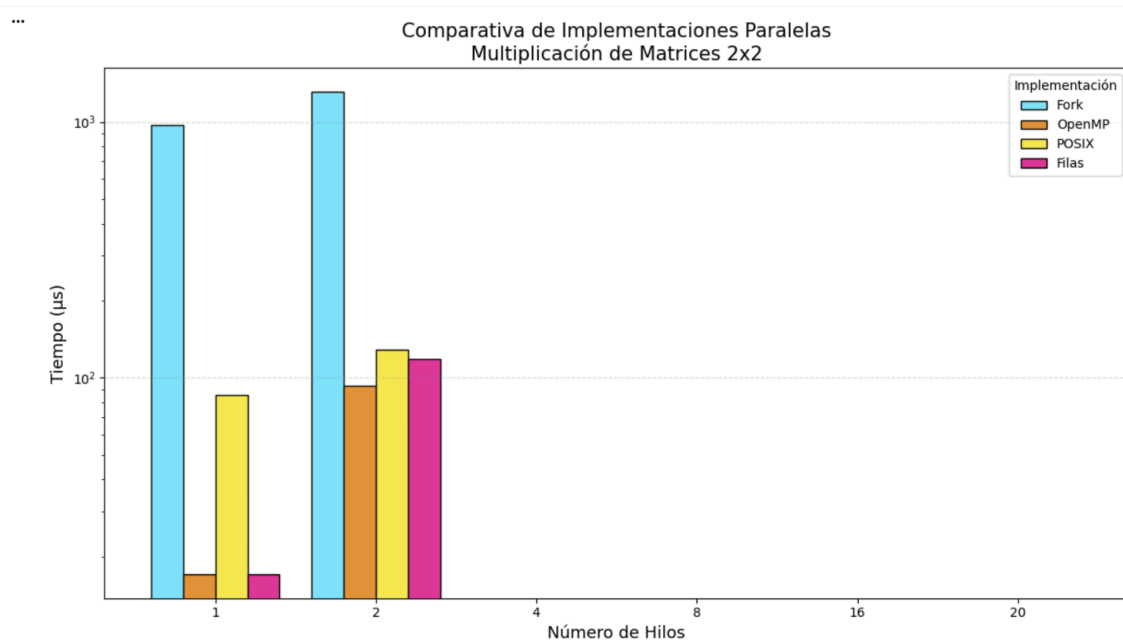
Para tamaños de matriz pequeños, el aumento de hilos genera ligeras variaciones, sin mejoras y con aumentos en el tiempo promedio. En matrices más grandes, principalmente al usar mayores cantidades de hilos el tiempo aumenta en gran medida, mostrando lo que hemos visto que el paralelismo y estos métodos de aprovechamiento de recursos consumen más recursos que los beneficios que traen en problemas con escalas bajas.

Archivo	Tamaño	Hilos	Promedio (µs)
MM_FilasOpenMP	2	1	17,1
MM_FilasOpenMP	2	2	118,07
MM_FilasOpenMP	10	1	14,07
MM_FilasOpenMP	10	2	115,97
MM_FilasOpenMP	10	4	182,97
MM_FilasOpenMP	10	8	284,27
MM_FilasOpenMP	20	1	15,73
MM_FilasOpenMP	20	2	115
MM_FilasOpenMP	20	4	188,77
MM_FilasOpenMP	20	8	283,2
MM_FilasOpenMP	20	16	452,4
MM_FilasOpenMP	20	20	510,4

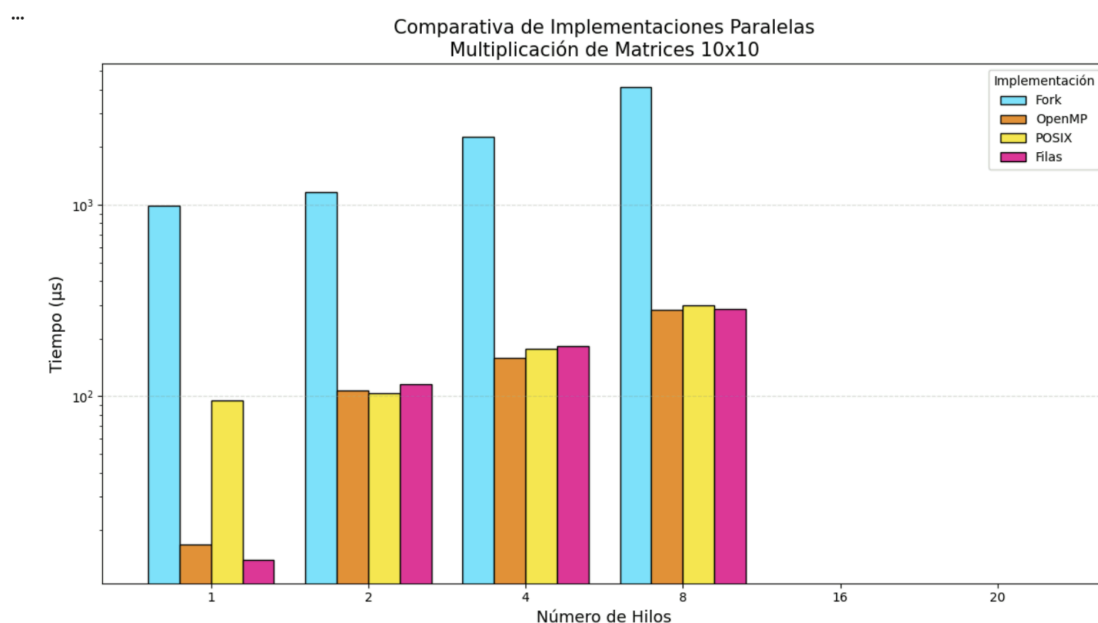
En esta versión se observa un comportamiento muy similar al de MM_ClasicaOpenMP, con ciertas diferencias en los tiempos. El rendimiento inicial con pocos hilos es excelente, pero al incrementar la cantidad de hilos el tiempo total aumenta.

Para todas las matrices, 2x2, 10x10 y 20x20 al aumentar el número de hilos aumentar el tiempo promedio, esto se debe al costo de gestión y sincronización de los hilos con respecto al tamaño del problema ya que se consumen más recursos en las operaciones de los hilos que el beneficio que trae usar estos. Asimismo, tiene mejores tiempos que Posix y Fork, junto a resultados parecidos a la programa clásico de OpenMP demostrando que es una de las mejores opciones.

A continuación, se muestran las gráficas de rendimiento para cada tamaño de matriz, donde se comparan las cuatro implementaciones paralelas en función del número de hilos utilizados. Estas visualizaciones permiten identificar el comportamiento, la eficiencia y la sobrecarga asociada a cada técnica de paralelismo.

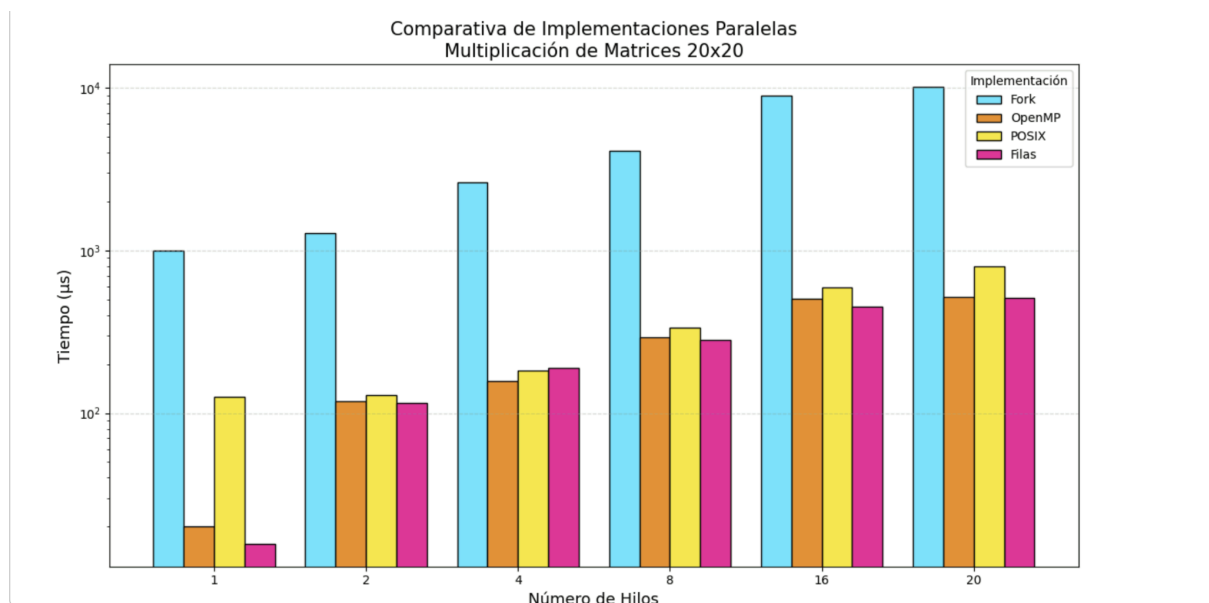


De la gráfica de comparativa de multiplicación 2x2 se puede concluir que el método Fork tiene un costo de tiempo de hasta el doble que su alternativa más cercana siendo esta Posix. Por otra parte, tanto el método de Filas como el de OpenMp clásica tienen un costo en tiempo bastante similar y considerablemente inferior que el de Posix dejándolos como la mejor opción. De igual manera, se evidencia que al aumentar el número de hilos las diferencias entre los distintos programas ejecutados se redujeron considerablemente pese a que Fork siendo claramente mayor, la distancia entre los dos de OpenMp y Posix si se acercó bastante.



De la gráfica de comparativa de multiplicación 10x10 se puede concluir que el método Fork tiene un costo de tiempo de hasta el doble que su alternativa más cercana siendo esta Posix en el caso de un solo hilo. Por otra parte, tanto el método de Filas como el de OpenMp clásica

tienen un costo en tiempo bastante similar siendo con un solo hilo mejor el de filas y con 2, 4 y 8 hilos el clásico. Cabe mencionar, que la distancia pese a que se mantiene constante siendo Fork el más demorado y Posix el segundo más lento se va reduciendo, los tiempos se van acercando entre mayor sea el número de hilos. Adicionalmente, en algunos casos como 2 hilos se demora más que Posix, pese a que en general es más rápido en ese caso el costo es incluso más perjudicial en manejo de recursos, en general todos tuvieron mejores resultados con un solo hilo.



De la gráfica de comparativa de multiplicación 20x20 se puede concluir que el método Fork tiene un costo de tiempo sumamente elevado respecto a los otros, su perseguidor más cercano es Posix en la mayoría de los hilos a excepción de dos. Por otra parte, tanto el método de Filas como el de OpenMp clásico tienen un costo en tiempo bastante similar siendo con un solo hilo, 8, 16 y 20 mejor el de filas y con 2 y 4 hilos el clásico. Cabe mencionar, que la distancia pese a que se mantiene constante siendo Fork el más demorado y Posix el segundo más lento se va reduciendo, los tiempos se van acercando entre mayor sea el número de hilos. Adicionalmente, entre más hilos el tiempo va aumentando más pero no es constante, en los casos de mayor cantidad de hilos como 16 y 20 el aumento es menor a comparación de casos como 4 y 8 hilos.

4. Conclusiones

Los resultados que se obtuvieron en los tres sistemas de cómputo evaluados, desde un MacBook Air 2015 con Intel Core i5 de dos núcleos y 8 GB de RAM, hasta los MacBook Pro 2023 con chip Apple M3 Pro de 11 núcleos y 18 GB de RAM, demuestran de manera clara y reproducible que el uso de paralelismo no mejora el rendimiento en problemas de baja granularidad computacional, como la multiplicación de matrices pequeñas.

En todos los casos y sistemas, el aumento del número de hilos o procesos incrementó sistemáticamente el tiempo de ejecución, evidenciando que el costo de creación, sincronización y coordinación de los hilos supera ampliamente el beneficio que aporta dividir el trabajo entre ellos.

Entre las implementaciones analizadas, OpenMP en su versión clásica y por filas se posicionó como el modelo más eficiente, registrando los menores tiempos de ejecución en configuraciones secuenciales con un solo hilo.

La versión mmFilasOpenMP mostró un comportamiento más equilibrado y escalable, con ligeras ventajas en las matrices de 20×20 gracias a una mejor calidad de datos y menor cantidad de fallos de caché, aunque la mejora no fue suficiente para compensar el costo de usar múltiples hilos.

POSIX Threads obtuvo un rendimiento medio, gestionando mejor la memoria compartida que Fork, pero con una sobrecarga visible en la sincronización.

Por su parte, Fork presentó el peor desempeño en todos los sistemas, con un incremento drástico del tiempo de ejecución a medida que se aumentaban los procesos, debido al alto costo de duplicación de memoria y de gestión de procesos independientes por parte del sistema operativo.

Por último, este taller demuestra una lección fundamental en sistemas operativos y computación paralela: el paralelismo solo resulta beneficioso cuando la carga de trabajo es lo suficientemente grande como para amortizar los costos de coordinación.

Modelos como OpenMP destacan por su eficiencia y facilidad de uso, POSIX representa una alternativa estable y equilibrada, mientras que Fork debe evitarse en tareas que requieren acceso compartido a memoria.

Los resultados, consistentes tanto en hardware moderno como en equipos más antiguos, evidencian que para problemas pequeños, la versión secuencial con un solo hilo sigue siendo la estrategia más eficiente.